Hochschule Konstanz
Fakultät Informatik

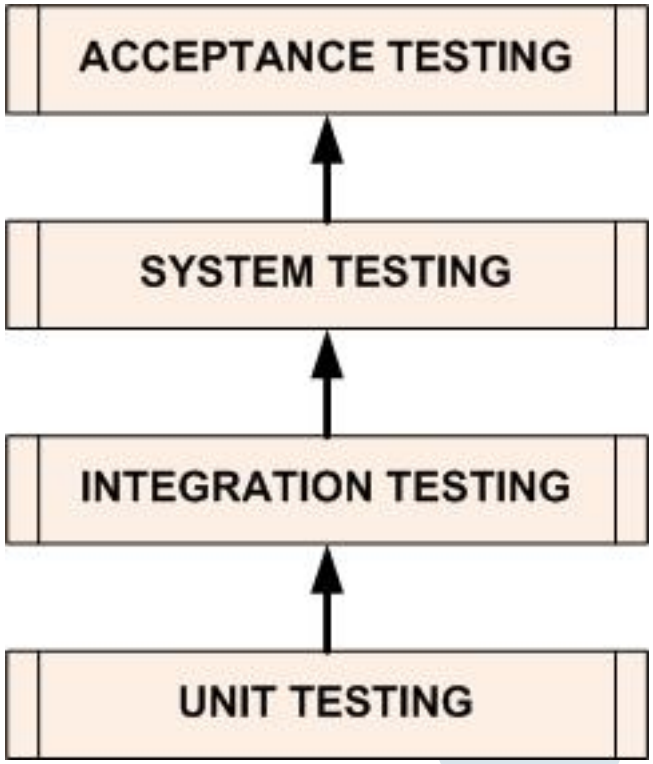
Software Architecture

Performance Testing with Gatling

Prof. Dr. Marko Boger

Types of Testing

- Unit Testing
  - Individual units of a software like functions, classes or components are tested in isolation.
- Integration Testing
  - Individual units are combined and tested as a group.
- System Testing
  - A complete, integrated system is tested.
- Acceptance Testing
  - A system and how it is used by its users is tested.



Unit Testing

Unit Testing is done during the development.

It focuses on the correct functionality of the smallest testable parts of an application, usually individual methods or functions.

It does not focus on non-functional aspects and does not put the application under strain.

Integration Testing

Integration Testing is a level of software testing where individual units are combined and tested as a group. The purpose of this level of testing is to expose faults in the interaction between integrated units. Test drivers and test stubs are used to assist in Integration Testing.

- Black Box Testing
  - The internal structure of the item being tested is not known to the tester.
- White Box Testing
  - The internal structure of the item being tested is known to the tester.

System Testing

- In System Testing the entire System as a whole is tested. Here the focus is usually not on the functional requirements of the system anymore, but rather on the not-functional aspects.
- An important form of System Testing is Performance Testing.

Acceptance Testing

Acceptance Testing is done after the system as a whole is functioning. But now it needs to be tested if all business requirements are met, and if the system is well suited for the intended user.

These tests are usually done by the client rather than the provider.

Performance Testing

- Performance testing is non-functional testing performed to determine how a system performs in terms of responsiveness and stability under a particular workload. It can also serve to investigate, measure, validate or verify other quality attributes of the system, such as scalability, reliability and resource usage.
- The goal is to find the bottleneck of the system and to fix it.

Types of Performance Testing

- Load Testing
- Stress Testing
- Volume Testing
- Endurance Testing
- Spike Testing

Load Testing

- Load testing is the simplest form of performance testing. A load test is usually conducted to understand the behavior of the system under a specific expected load. This load can be the expected concurrent number of users on the application performing a specific number of transactions within the set duration.

Stress Testing

- Every system has a capacity limit. When the load goes beyond the limit, the web site starts responding very slowly and even produce errors.
- The purpose of the stress testing is to find the capacity limit of the system. We verify at which point of time the system degrades or fails. This is usually done by increasing the user load in the system.

Volume Testing

- Volume testing is non-functional testing that is performed as part of performance testing where the software is subjected to a huge volume of data. It is also referred to as flood testing.

Endurance Testing

- This type of testing is used to check that the system can withstand the load for a long time or large number of transactions.
- Tests will be performed with defined set of concurrent users for a prolonged period of time, say for example 5 to 10 hours or 2 to 3 days.

Spike Testing

- Spike testing is done by suddenly increasing the load generated by users by a very large amount and observing the behavior of the system. The goal is to determine the performance degrade, system failure or how it will be able to handle dramatic changes in load.

Performance Testing Tools

- Load Runner, commercial load testing tool from HP
- JMeter, an open source tool from Apache
- RPT, commercial load test tool from IBM
- NeoLoad, commercial, for Windows, Linux, Solaris
- Microsoft Visual Studio Team System 2010, commercial, for Windows, which includes Load Test Analyzer and Load Test Monitor tools.
- OpenLoad, commercial load testing tool and hosted service
- OpenSTA, an open source tool
- PureLoad, commercial, multiplatform load testing tool
- PushToTest TestMaker, an open source testing framework (load testing and more)
- QEngine, free and commercial, from AdventNet (free edition supports 5 virtual users)
- SQLQueryStress Performance Testing Tool, free, for testing SQL Server StressIT, commercial and free
- The Grinder, an open source tool
- Flood, open source from and for Apache
- WAPT, Web Application Testing tool, a commercial product, for Windows
- WatchMouse, commercial hosted load testing service
- WebKing, commercial, multiplatform
- WebServer Stress Tool, commercial and free, from Paessler

JMeter

- It is an Open source tool.
- Can load and performance test many different server types:
  - Web - HTTP, HTTPS
  - SOAP
  - Database via JDBC
  - LDAP
  - JMS
  - Mail - POP3(S) and IMAP(S)
- User friendly GUI Design compared to other tools.
- Full multithreading framework allows concurrent sampling by many threads and simultaneous sampling of different functions by separate thread groups.
- Caching and Cookies can be enabled and it impact as such as it is executing in the web browser.
- Controllers are configurable and can monitor the server performance.
- Test results are more reliable compared to other open source tools.
- Test results can be captured in various formats like summary report, Graph, Aggregate report, Aggregate graph, Results in tree and Results in Table.

Elements of Test Plans

- Thread Group
  - Setup number of threads
  - Set up ramp up period
  - No. of times test executes
- Controllers
  - Sampler (Send Request to Server)
  - Logical Controller (Customize logic to send request)
- Listener
  - Graph Result
  - View Results Tree and many more.
- Timers
  - Delay next request for certain amount of time
- Assertions
  - Allow you to assert facts about responses received from HTTP request
- Configuration Elements
  - Allow to configure settings
- Preprocessor
  - Execute prior to sampler request
- Post Processor
  - Execute some action after sampler request

Apache JMeter (2.6 r1237317)

File Edit Search Run Options Help

0 / 0

Test Plan
WorkBench

Test Plan

Name: Test Plan
Comments:

User Defined Variables

Name:	Value

Add | Add from Clipboard | Delete | Up | Down

☐ Run Thread Groups consecutively (i.e. run groups one at a time)

☐ Functional Test Mode (i.e. save Response Data and Sampler Data)

Selecting Functional Test Mode may adversely affect performance.

Add directory or jar to classpath | Browse... | Delete | Clear

Library

98% 5:50 PM 11/7/2012

Apache JMeter (2.6 r1237317)

File Edit Search Run Options Help

Test Plan
Work

Add
Paste Ctrl-V
Reset Gui
Open...
Merge
Save Selection As...
Save Node As Image Ctrl-G
Save Screen As Image Ctrl+Shift-G
Enable
Disable
Toggle Ctrl-T
Help

Threads (Users)
Test Fragment
Config Element
Timer
Pre Processors
Post Processors
Assertions
Listener

Thread Group
setUp Thread Group
tearDown Thread Group

User Defined Variables

Name:	Value

Add | Add from Clipboard | Delete | Up | Down

Run Thread Groups consecutively (i.e. run groups one at a time)

Functional Test Mode (i.e. save Response Data and Sampler Data)

Selecting Functional Test Mode may adversely affect performance.

Add directory or jar to classpath | Browse... | Delete | Clear

Library

Controller.jmx (D:\Activity\Demo\Controller.jmx) - Apache JMeter (2.6 r1237317)

File Edit Search Run Options Help

Test Plan
Thread Group

Add
Cut Ctrl-X
Copy Ctrl-C
Paste Ctrl-V
Duplicate Ctrl+Shift-C
Reset Gui
Remove Delete
Open...
Merge
Save Selection As...
Save Node As Image Ctrl-G
Save Screen As Image Ctrl+Shift-G
Enable
Disable
Toggle Ctrl-T
Help

Logic Controller
Config Element
Timer
Pre Processors
Sampler
Post Processors
Assertions
Listener

ForEach Controller
If Controller
Include Controller
Interleave Controller
Loop Controller
Module Controller
Once Only Controller
Random Controller
Random Order Controller
Recording Controller
Runtime Controller
Simple Controller
Switch Controller
Throughput Controller
Transaction Controller
While Controller

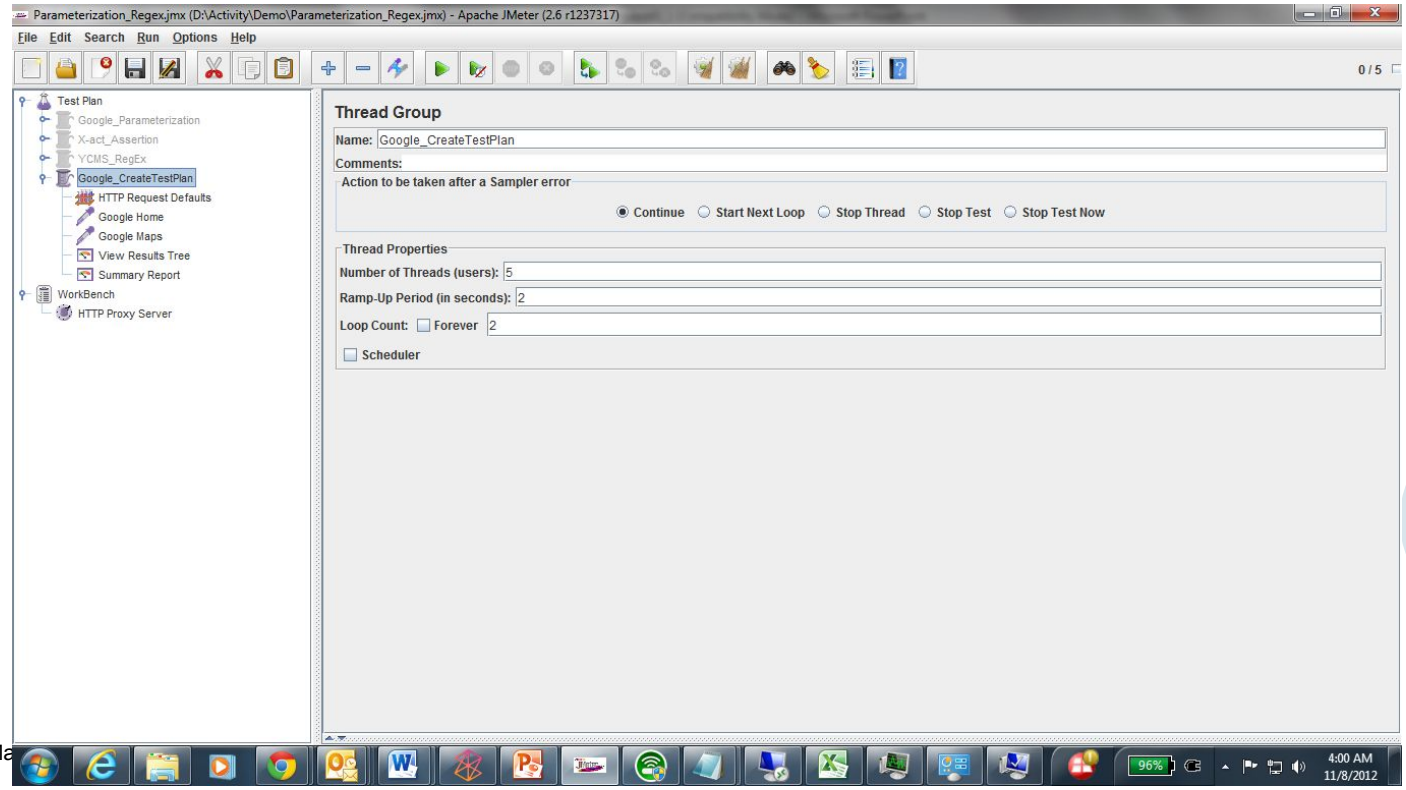
error
Continue Start Next Loop Stop Thread Stop Test Stop Test Now

/ycms/js/vag-rounded-light_700.font.js
HTTP Header Manager
/ycms/js/cufon.init.js
HTTP Header Manager
/ycms/images/mainbanner_huddlexs_new_discount.
HTTP Header Manager
/ycms/js/jquery-1.7.1.min.js
HTTP Header Manager
/ycms/images/mainbanner_eclipse_new.jpg
HTTP Header Manager
/ycms/js/vag-rounded_500-vag-rounded_500.font.js
HTTP Header Manager

5:45 PM
11/7/2012

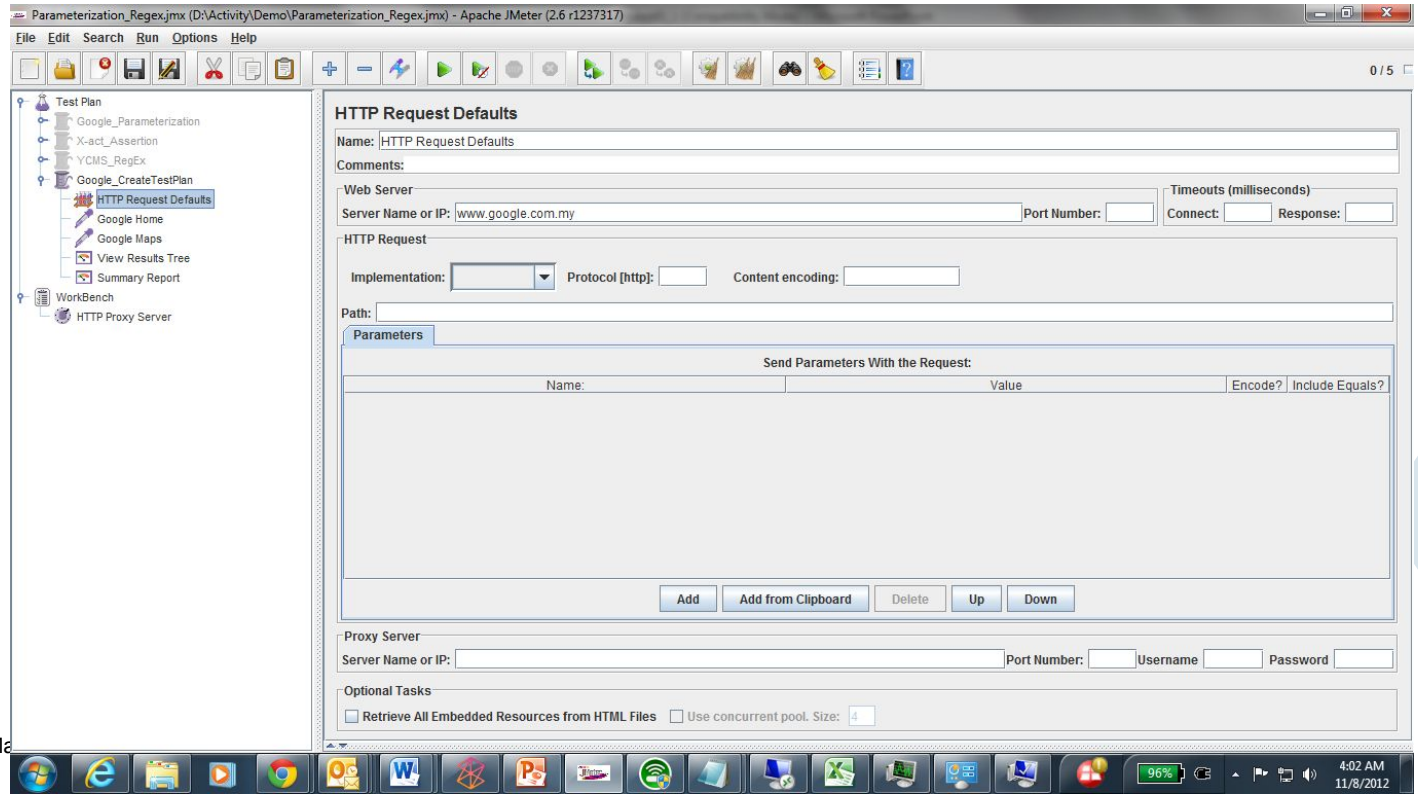
Adding Users

- 5 users send 2 requests on cfminds.com and repeat it twice. (5 users x 2 requests x 2 repeat = 20 requests)
- Right click on test plan >> Add >> Thread >> Thread Group



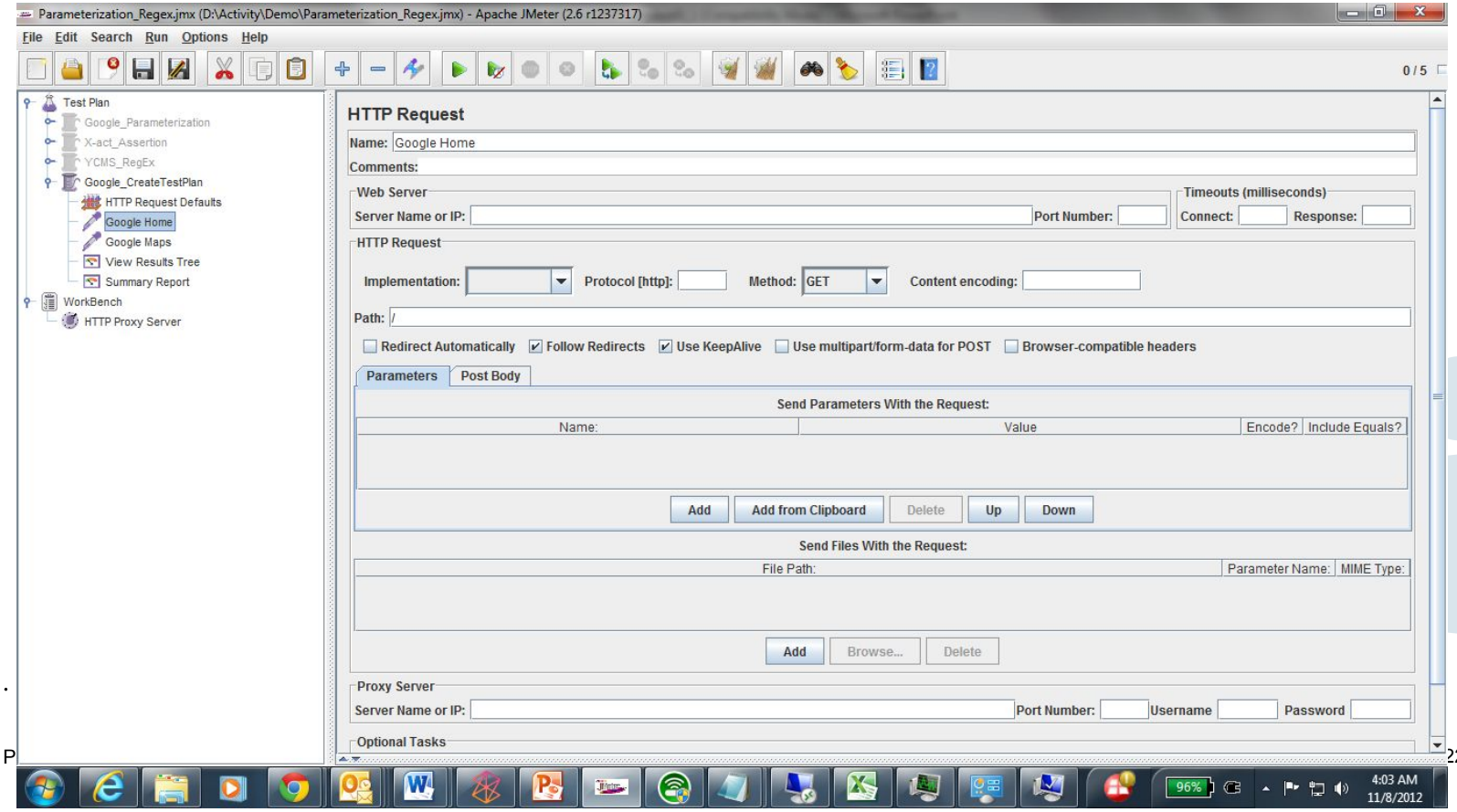
Add a default http request

- You can add default http request setting from ADD >> Config Element >> HTTP request default.
- This will let you define default http parameter for every request.



Add HTTP Request element

- You can add it from ADD> Sampler >> HTTP Request

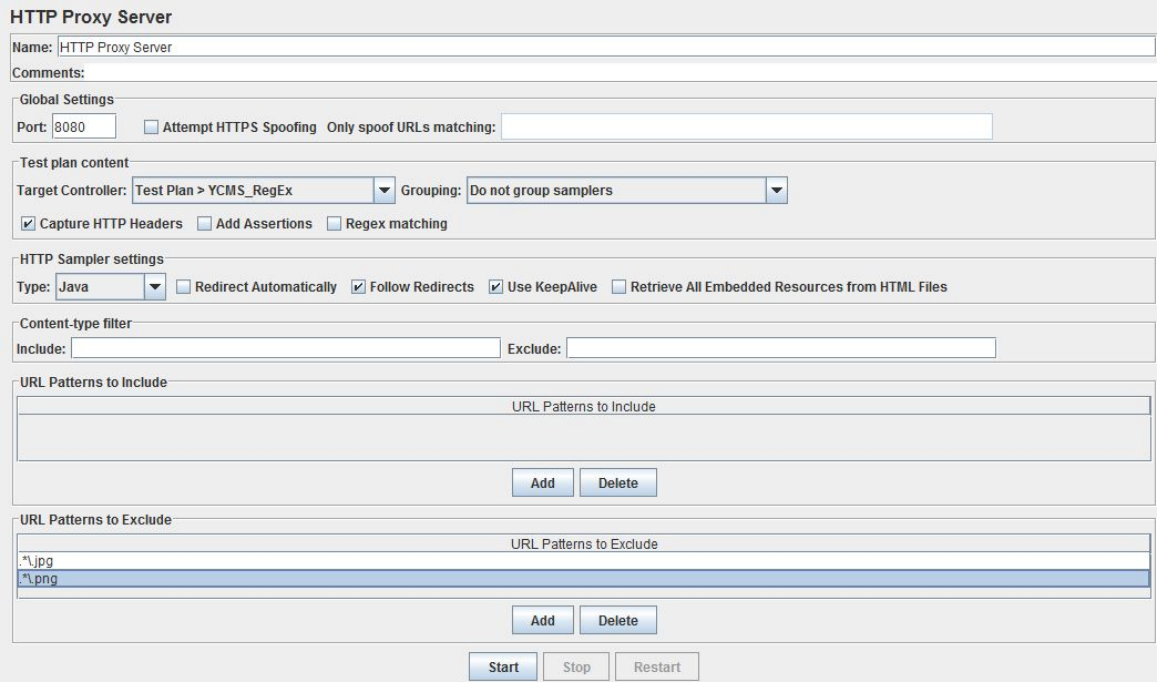


Add a Listener to view Results

- Listner responsible for storing all of the results of our HTTP request and presenting in Visualize mode.

Record your Test Plan

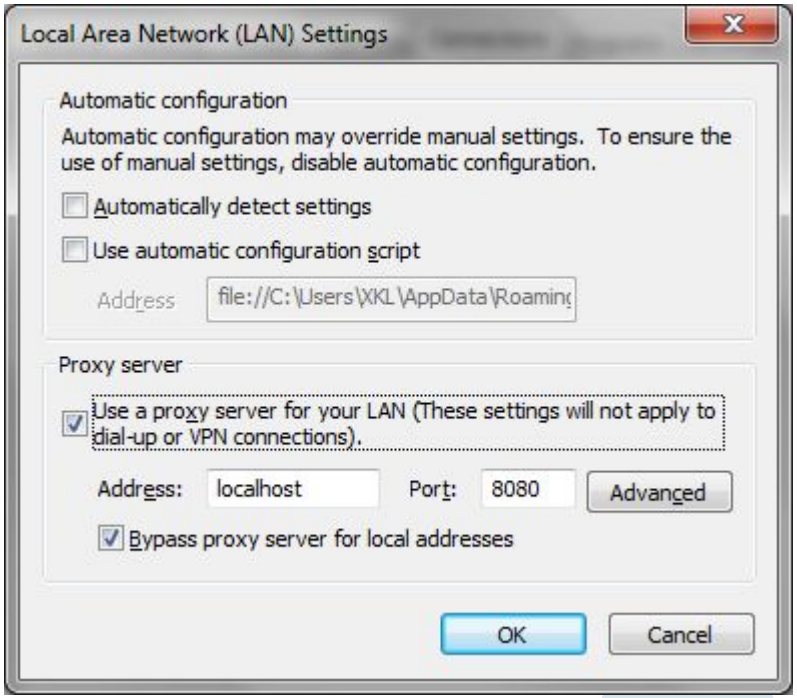
- JMeter Proxy can be used to record all requests sent to the server.
- Create test plan with default http testing
- Add HTTP Proxy Server in Workbench node.
  - Define port# of proxy server
  - Define URL Pattern to include/exclude
    - o .*\.jpg
    - o .*\.js
    - o .*\.png
    - o .*\.gif
    - o .*\.jsp
    - o .*\.css
    - o .*\.txt
- Start Proxy server.



Record your Test Plan

- Setup Proxy server on your browser.

- Start Recording using the Web Browser.
  - All the Requests will be captured in the Test plan.



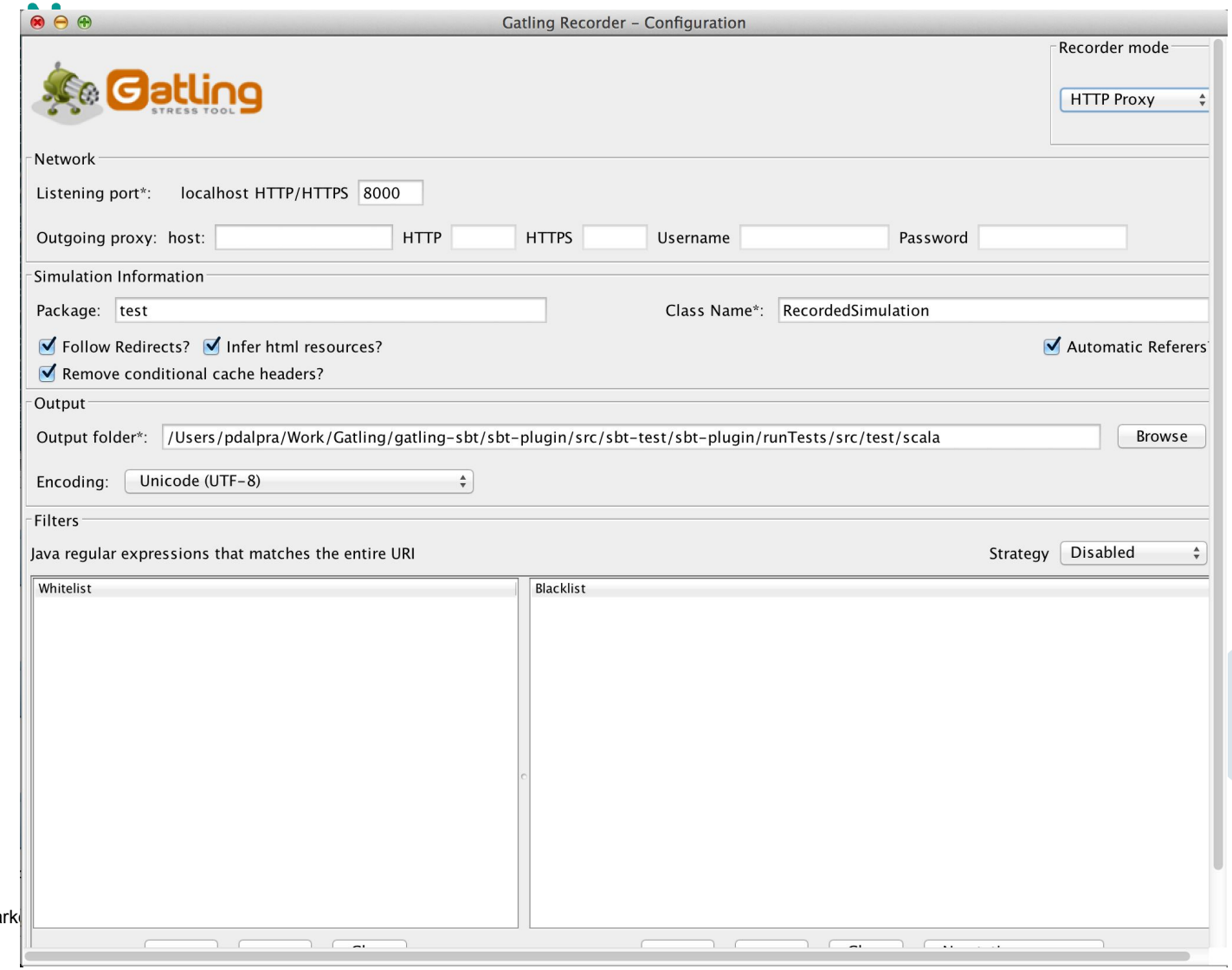
Gatling

- Gatling is an open-source load and performance testing framework based on Scala, Akka and Netty.

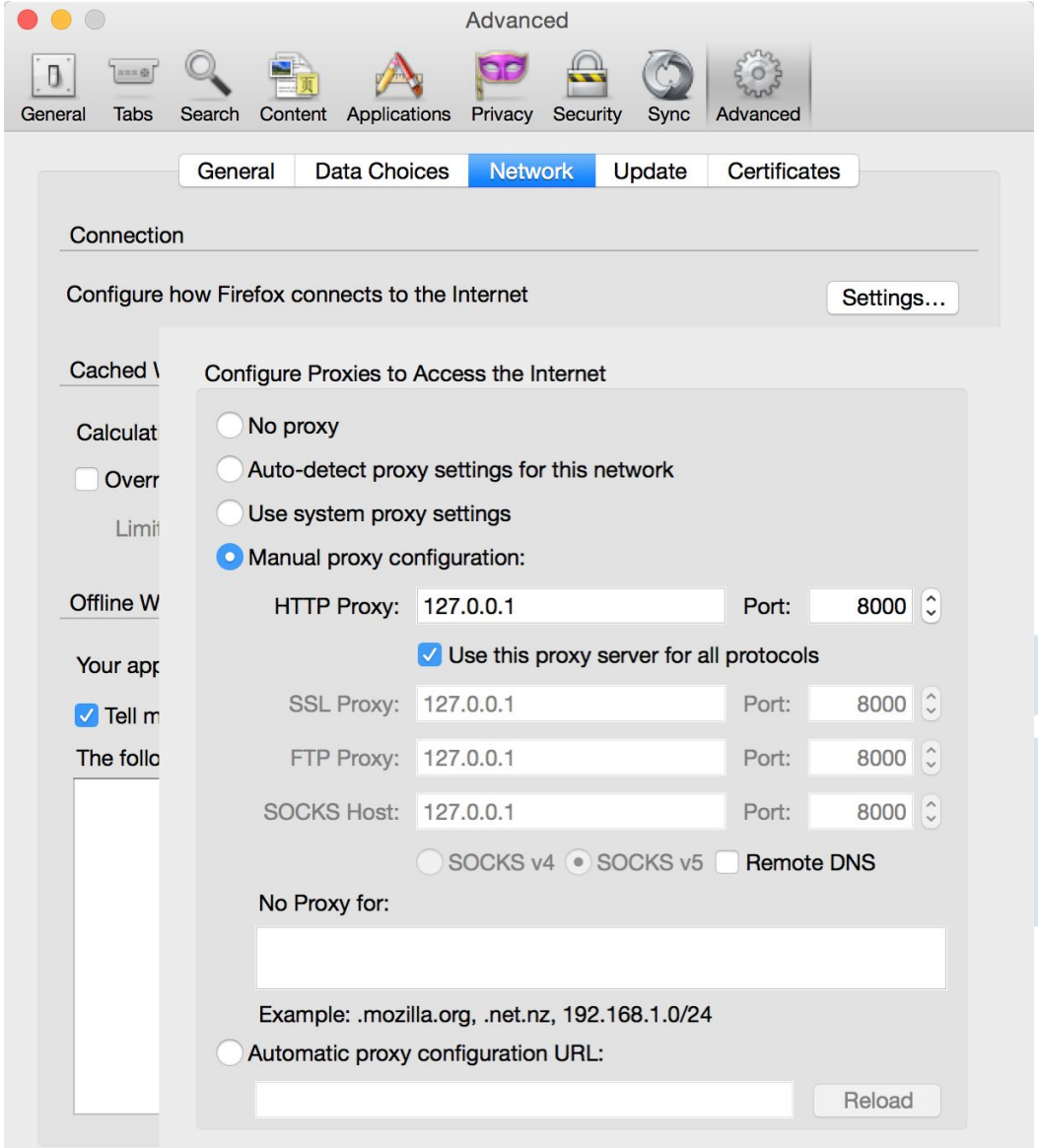
Gatling

- Install
  - Download zip, unzip into any folder (not under C:/Programs)
  - Start the Gatling recorder
    - o Folder/bin/recorder.sh
  - Configure your browser to use a proxy
    - o Localhost:8000 or 127.0.0.1 Port 8000
  - Start your Microservice
  - Start the recorder
  - Use your Microservice from the browser
  - Stop the recorder
  - Open the file
    - o Folder/user-files/simulations/RecordedSimulation.scala

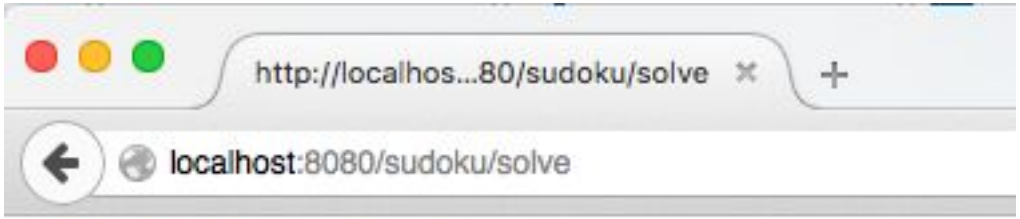
Gatling Recorder



Settings for the Proxy (Firefox)



Use the Application



HTWG Sudoku

```
+-------+-------+-------+
| 1 5 6 | 3 9 2 | 4 7 8 |
| 9 7 4 | 8 5 1 | 3 2 6 |
| 3 8 2 | 7 6 4 | 5 9 1 |
+-------+-------+-------+
| 5 6 7 | 4 3 9 | 8 1 2 |
| 2 9 3 | 1 7 8 | 6 4 5 |
| 8 4 1 | 6 2 5 | 7 3 9 |
+-------+-------+-------+
| 6 2 9 | 5 4 7 | 1 8 3 |
| 7 1 5 | 2 8 3.| 9.6 4 |
| 4 3 8 | 9 1.6 | 2 5 7 |
+-------+-------+-------+
```

H T
W
G

RecordedSimulation.scala

BasicSimulation.scala | RecordedSimulation.scala ×

```scala
import scala.concurrent.duration._

import io.gatling.core.Predef._
import io.gatling.http.Predef._
import io.gatling.jdbc.Predef._

class RecordedSimulation extends Simulation {

    val httpProtocol = http
        .baseURL("http://localhost:8080")
        .inferHtmlResources()
        .acceptHeader("text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8")
        .acceptEncodingHeader("gzip, deflate")
        .acceptLanguageHeader("en-US,en;q=0.5")
        .userAgentHeader("Mozilla/5.0 (Macintosh; Intel Mac OS X 10.11; rv:36.0) Gecko/20100101 Firefox/36.0")



    val uri1 = "http://localhost:8080/sudoku"

    val scn = scenario("RecordedSimulation")
        .exec(http("request_0")
            .get("/sudoku/new"))
        .pause(14)
        .exec(http("request_1")
            .get("/sudoku/random"))
        .pause(14)
        .exec(http("request_2")
            .get("/sudoku/111"))
        .pause(8)
        .exec(http("request_3")
            .get("/sudoku/solve"))
        .pause(7)
        .exec(http("request_4")
            .get("/sudoku/new"))
        .pause(11)
        .exec(http("request_5")
            .get("/sudoku/random"))
        .pause(5)
        .exec(http("request_6")
            .get("/sudoku/solve"))

    setUp(scn.inject(atOnceUsers(1))).protocols(httpProtocol)
}
```

⊗ 0 ⚠ 0 Zeile 1, Spalte 1 Tabulatorgröße: 4 UTF-8 LF Scala

Run a Simulation

- Set GATLING_HOME
  - GATLING_HOME = file: /Users/mboger/Documents/gatling-charts-highcharts-bundle-2.2.5/
- Run $GATLING_HOME/bin/gatling.sh
- Select the RecordedSimulation
- Open the generated Report in a Browser

- Then change the code for your simulation

file:///Users/rfriedman/Downloads/report-files%206/index.html

Gatling STRESS TOOL

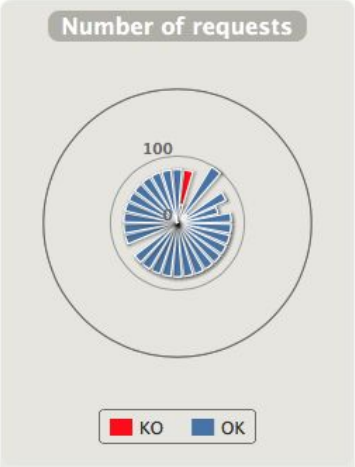
plexifysimulation220

GLOBAL | DETAILS

2016-07-05 10:30:56 -04:00, duration : 50 seconds Redline13_Executed

- Active Users
- Requests / sec
- Responses / sec

> Global Information



STATISTICS (Click here to show more)

Expand all groups | Collapse all groups

Requests	Executions					Response Time (ms)							
	Total	OK	KO	% KO	Req/s	Min	50th pct	75th pct	95th pct	99th pct	Max	Mean	Std Dev
Global Information	2180	2128	52	2%	18.319	2	128	293	1207	1999	4147	272	420
request_0	80	80	0	0%	0.672	5	699	1419	1942	2050	2392	942	584
request_...direct 1	80	28	52	65%	0.672	23	1538	2001	2700	3055	4147	1516	810
favicon....direct 1	1	1	0	0%	0.008	477	477	477	477	477	477	477	0
brand?fo...direct 1	98	98	0	0%	0.824	12	116	381	596	626	630	218	191
brand?fo...direct 2	18	18	0	0%	0.151	19	130	278	320	342	348	167	109
google_cse	80	80	0	0%	0.672	12	278	615	638	648	655	307	281
google_c...direct 1	62	62	0	0%	0.521	8	94	113	130	172	179	78	42
youtube Redirect 1	80	80	0	0%	0.672	12	121	137	162	255	280	92	61
request_3	80	80	0	0%	0.672	3	7	34	101	156	160	25	34
request_6	80	80	0	0%	0.672	5	35	78	280	410	823	76	115

Performance Metrics

Performance has several meanings

- Throughput
  - Amount of work done during a unit of time
- Response Time
  - Amount of time it takes the system to process a request
- Scalability
  - Describes behavior of the system as a function of loading conditions
- Memory consumption

Before you optimize

"Premature optimization is the root of all evil."

- Donald Knuth

- Reasons not to optimize
  - If your code already works, optimizing it is a sure way to introduce new, and possibly subtle, bugs
  - Optimization tends to make code harder to understand and maintain
  - Some of the techniques increase speed by reducing the extensibility of the code
  - Optimizing code for one platform may actually make it worse on another platform
  - A lot of time can be spent optimizing, with little gain in performance, and can result in obfuscated code

Alternatives to Optimization

- Moores law
  - Hardware performance doubles every 18 months
  - So the problem may be obsolete by the time it becomes relevant
- Hardware
  - Better hardware can solve a problem for lower cost
- Compiler improvements
  - Byte code optimization
  - JIT compiler, compile byte code to native code
- Library improvements
  - Performance improvement of factor 50 between Java 1.0 and Java 1.6

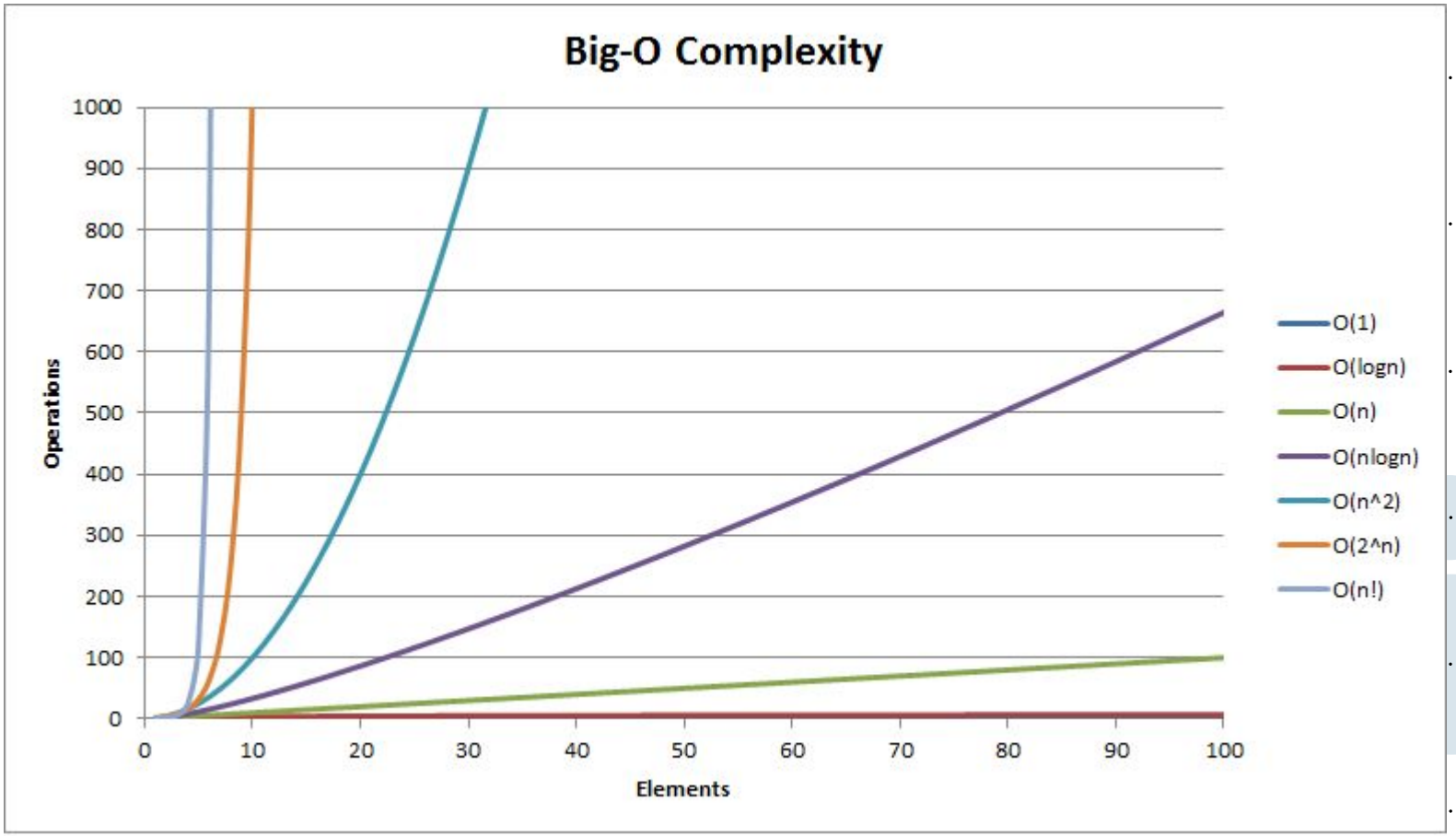
Why Optimize

- Limited Resources
  - CPU cycles
  - Memory
  - Network bandwidth
- In Business Applications, usually speed is the main problem
- In Embedded Systems, memory is the main problem
- In Distributed Systems, network bandwidth can be a big problem

Choices

- Improve Algorithms
  - O()-notation
  - o Sort-Algorithms
    - > Insertion sort, Bubble sort: $O(n^2)$,
    - > Quicksort: $O(n \log(n))$ on average
- Use native code
  - C code can be a lot faster
  - o Platform independence lost
  - o Harder to maintain
- Improve the general code
  - 90% of the time is spent in 10% of the code
  - Only measuring can point to these 10%
  - Optimizing the other 90% of the code is pointless.

Effects of Improving the Complexity of Algorithms



General Rule

- First build the system
- Then make it fast
- Bottlenecks are not where you expect them
- Only improve where you get a benefit
- Improving performance in the small usually means a decrease in clarity of design
- Large performance improvements are a question of the underlying architecture

Performance from Tests

Tests are pretty good for
Performance testing

- Fixed
- Repeatable
- Based on small Units
- To increase accuracy, repeat tests in a loop

This approach also has several disadvantages

- It measures time, not CPU cycles
- Garbage Collection and other tasks have an influence
- Memory is not taken into account

Management Perspective

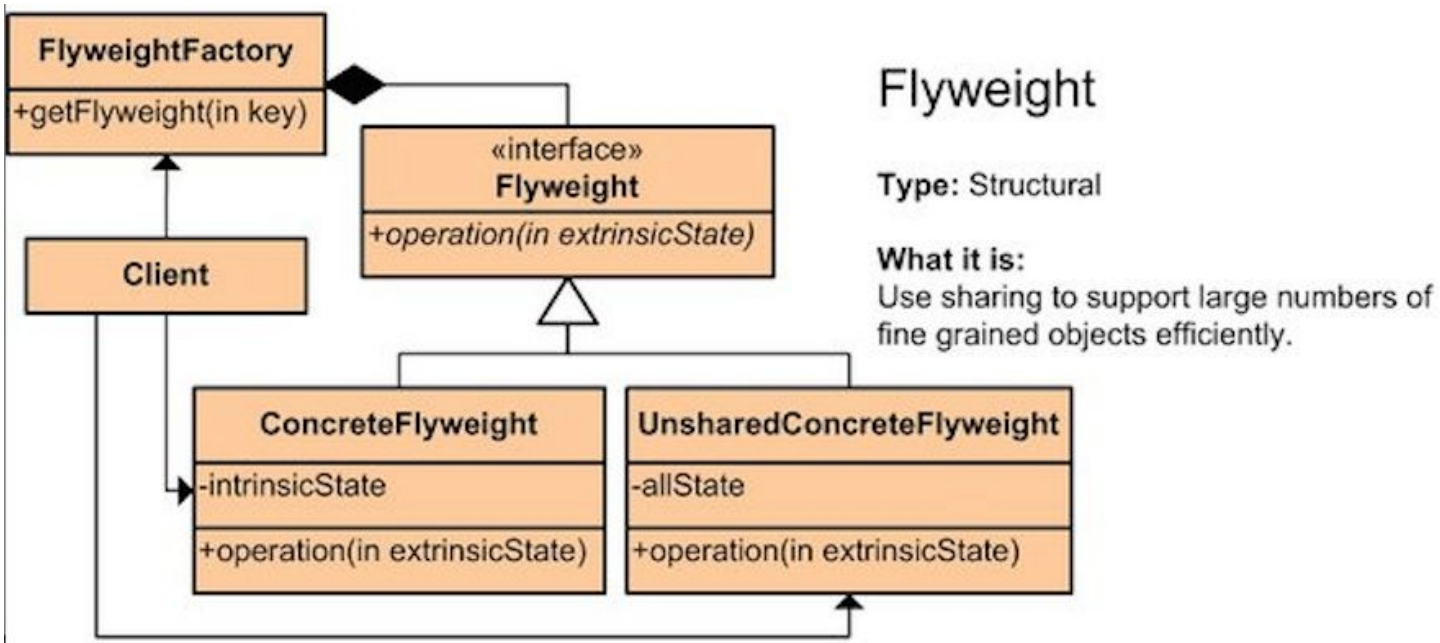
- Include budget for performance management.
- Create internal performance experts.
- Set performance requirements in the specifications.
- Include a performance focus in the analysis.
- Require performance predictions from the design.
- Create a performance test environment.

Performance Tuning

- Some design patterns can be used to optimize performance
  - Flyweight reduces memory consumption.
  - Object Pool reduces memory consumption.
  - Prototype increases object creation speed.
  - Proxy increases communication speed.
- Caching data in main memory instead of fetching from a database has a huge performance impact.
- Eager or lazy loading or creation can move performance costs in time to where the cost is less painful

Flyweight

The Flyweight pattern describes how to share objects to allow their use at fine granularities without prohibitive cost. Each "flyweight" object is divided into two pieces: the state-dependent (extrinsic) part, and the state-independent (intrinsic) part. Intrinsic state is stored (shared) in the Flyweight object. Extrinsic state is stored or computed by client objects, and passed to the Flyweight when its operations are invoked.



Example of Flyweight

The Flyweight uses sharing to support large numbers of objects efficiently. Modern web browsers use this technique to prevent loading same images twice. When the browser loads a web page, it traverses through all images on that page. The Browser loads all new images from the Internet and places them in the internal cache. For already loaded images, a flyweight object is created, which has some unique data like position within the page, but everything else is referenced to the cached one.

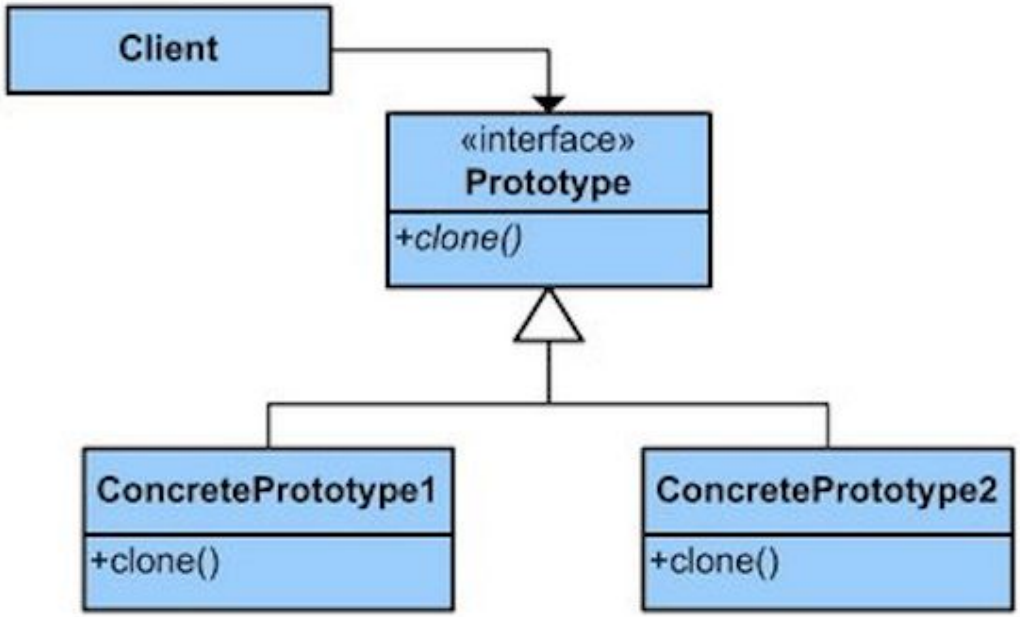
Prototype

Prototype pattern avoids the „new“ creation of an object and instead creates a clone of an existing one. Cloning can be much faster than creation.

Prototype

Type: Creational

What it is:
Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.



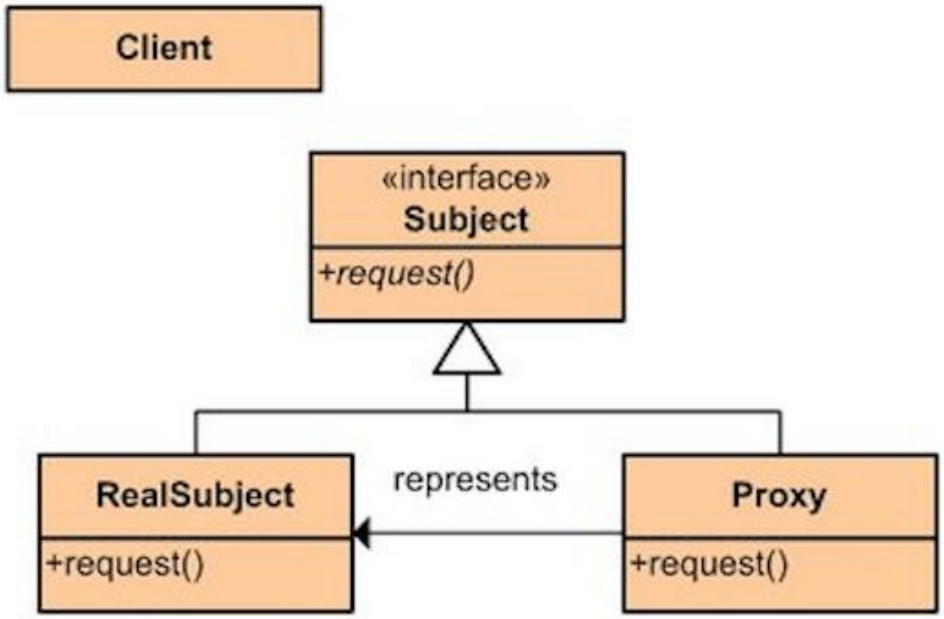
Proxy

A Proxy can represent a large or in some sense resource-hungry object. It can avoid the expense of the resources until it is actually needed.

Proxy

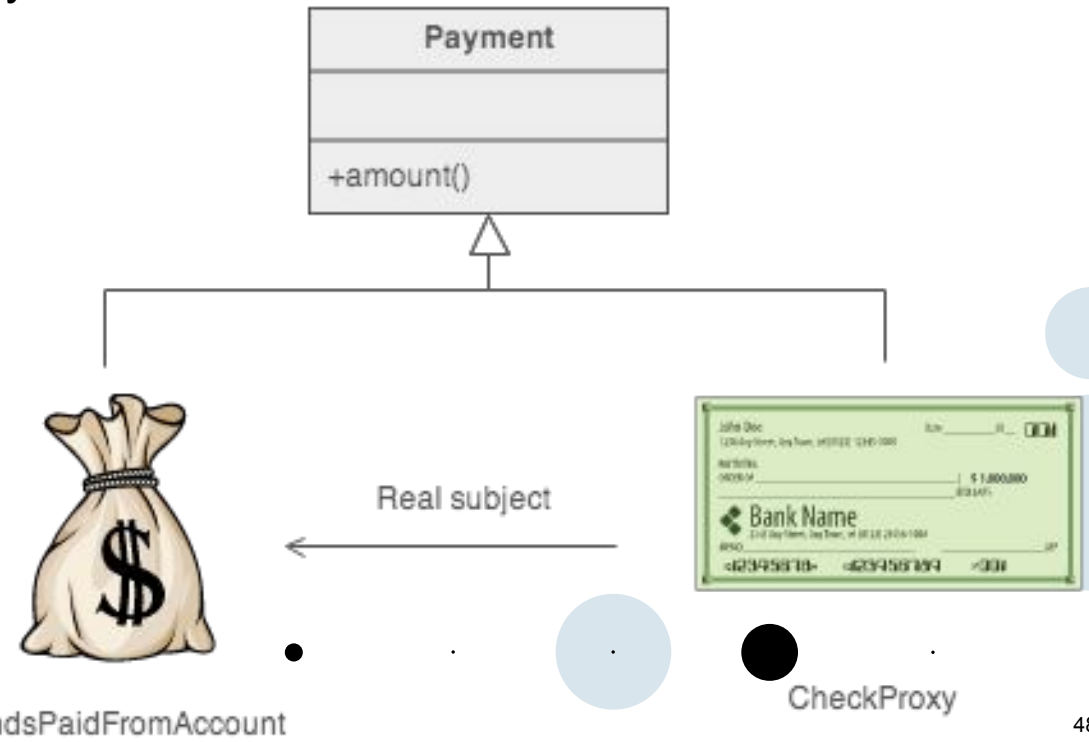
Type: Structural

What it is:
Provide a surrogate or placeholder for another object to control access to it.



Example for Proxy

The Proxy provides a surrogate or place holder to provide access to an object. A check or bank draft is a proxy for funds in an account. A check can be used in place of cash for making purchases and ultimately controls access to cash in the issuer's account.



Task 9

Generate a Performance-Test-Script using the Gatling Recorder.

Optimize the generated script by hand.

Analyse the report and optimize your application code.

Repeat the performance test and show the improvement.